

Automatização de Cenários "Cyber Range"

Nuno Bento - a48172

Trabalho realizado sob a orientação de

Prof. Tiago Pedrosa

Prof. Jorge Loureiro

Licenciatura em Engenharia Informática

2025-2026

Automatização de Cenários "Cyber Range"

Relatório da UC de Projeto
Licenciatura em Engenharia Informática
Escola Superior de Tecnologia e Gestão

Nuno Bento - a48172

2025-2026

Acknowledgments

The fact that I am writing this report means that my academic journey is almost coming to an end. During these years, I could not reach this stage without thanking my family and friends, who have always been present and encouraged me to be better and to do better.

I would also like to thank my supervisors, Professor Tiago Pedrosa and Professor Jorge Loureiro, for their guidance and continuous support during this project.

Abstract

Cybersecurity training requires realistic environments where vulnerabilities and attacks can be safely explored.

Ludus is an open source cyber range platform built on top of Proxmox. It allows the creation of multi virtual machines environments using configuration files. It simplifies the deployment of virtual networks and templates and supports automation. It is especially useful for managing complete ranges with routers, VLANs and multiple machines.

SecGen is an open source tool designed to automatically generate vulnerable machines for cybersecurity training. It creates intentionally insecure systems by installing vulnerable services and configurations. These machines can then be used for penetration testing and defensive exercises.

Although both tools are powerful, they focus on different aspects. Ludus is strong in infrastructure orchestration and range management. SecGen is strong in automated vulnerability generation. However, neither tool alone provides a complete solution that combines network orchestration with concurrent automated scenario generation.

This creates an opportunity to integrate both tools in order to develop a more complete and scalable cyber range platform. The objective of this project is to bridge this gap by combining the infrastructure management capabilities of Ludus with the vulnerability generation capabilities of SecGen.

Keywords: Secgen, Ludus, Cyber Range, Management.

Contents

1	Introduction	2
1.1	Objectives	3
1.2	Document Structure	5
2	Context and Tools	6
2.1	Technologies and Tools	7
2.1.1	Virtualization Plataforms	7
2.1.2	Tools	9
2.1.3	Programming Languages	11
2.1.4	Virtual Private Network (VPN) - Wireguard	12
3	Approach	14
3.1	Proposed Solution Architecture	15
3.2	Core Operational Workflow	16
4	Implementation	17
4.1	Ludus Installation on Proxmox	17
4.2	Template Build and Range Deployment	18
4.2.1	Wireguard	19
4.3	SecGen Installation on Ubuntu 20.04 VM	20
4.4	Adapting Vagrant Proxmox Provider	21
4.5	Critical Source Code Modifications	21

4.5.1	Fix 1: API Endpoint Handling	21
4.5.2	Fix 2: SSL Verification	22
4.5.3	Fix 3: Force POST	23
4.5.4	Fix 4: Skip Forwarded Port Logic	23
4.5.5	Fix 5: ACL Permissions	24
4.5.6	Fix 6: Full Clone	24
4.6	Integration Script Development	25
5	Tests	26
5.1	Ludus Template Preparation	26
5.2	Vulnerable Machine Generation with SecGen	28
5.3	Conversion to Template	29
5.4	Range Deployment with Vulnerable Machines	29
5.5	Discussion	30
6	Conclusions	32
A	Original Project Proposal	A2
B	Other Appendices	B1
B.1	Ludus	B1
B.1.1	config.yml	B1
B.1.2	ludus-range-config.yml	B2
B.2	Secgen	B2
B.2.1	Vagrantfile.proxmox.ludus.template	B2
B.2.2	script.sh	B4
B.2.3	vagrant-proxmox.gemspec	B7
B.2.4	connection.rb	B9
B.2.5	config_clone.rb	B21
B.2.6	clone_vm.rb	B25

List of Tables

5.1	Ludus range information.	30
-----	----------------------------------	----

List of Figures

5.1	Overview of the Proxmox node (pve).	27
5.2	Range of user JD.	31

Chapter 1

Introduction

Cybersecurity has become one of the most important areas in Information Technology. Organizations, universities and governments need professionals who are able to understand attacks, analyze vulnerabilities and respond to incidents in a safe and controlled way. For this reason, practical training environments are essential. However, testing real attacks in production systems is dangerous and not acceptable. Because of that, cyber ranges have become an important solution.

This project focuses on the design and implementation of a cyber range environment by integrating two open source tools, Ludus and SecGen. The main idea is to combine the strengths of both tools in order to automatically generate vulnerable controlled environments, consisting of vulnerable virtual machines inside isolated networks. The result is a platform that can be used for cybersecurity training and experimentation.

Cyber ranges are virtual environments that simulate real networks, systems and services. They allow students and professionals to practice offensive and defensive cybersecurity techniques without affecting real infrastructures. These platforms are widely used in education and technical training.

Proxmox Virtual Environment is an open source virtualization platform based on KVM technology. It allows the creation and management of virtual machines and containers through a web interface and command line tools. Proxmox provides features such as virtual networking, storage management, snapshots and template cloning, making it

suitable for laboratory environments and infrastructure testing.

There are several tools available to support this types of environments. Ludus is a platform built on Proxmox that allows the creation of virtual ranges using templates and configuration files. It is designed to simplify the deployment of complex virtual networks.

SecGen, on the other hand, focuses on autogenerated vulnerable machines for penetration testing and cybersecurity exercises.

Although both tools are powerful, they were not originally designed to work together. Ludus is responsible for creating and managing ranges, while SecGen was mainly developed to work with hypervisors (virtualization tools). This project is based on the original proposal presented in Appendix A and aims to integrate these two tools in a single functional solution.

An artificial intelligence (ChatGPT)[1] was used occasionally during this project. It was mainly used to review small parts of the written text and to assist in resolving some technical errors encountered during development.

1.1 Objectives

The main objective of this project is to integrate Ludus and SecGen in order to create a functional cyber range platform capable of generating vulnerable machines inside isolated virtual environments.

The project has the following goals:

- Install and configure a Proxmox server as the virtualization platform.
- Install and configure Ludus to create and manage ranges.
- Implement SecGen to use Proxmox as a virtualization provider.
- Automate the integration process of both solutions.
- Validate the integration by successfully creating customizable cyber ranges.

Project Timeline

The development of this project followed a structured and progressive approach. Due to the technical complexity of integrating different open source tools, the work was divided into several phases. Each phase focused on solving a specific part of the problem, gradually leading to the final integrated solution.

The first phase consisted of environment preparation. This included the installation and configuration of Proxmox virtualization platform and the installation of Ludus on the Proxmox. During this stage, templates were built, storage configurations were adjusted and initial range deployments were tested. This phase was essential to ensure that the infrastructure layer was stable and fully functional.

The second phase focused on the installation of SecGen and compatibility testing. Since SecGen was not fully compatible with the Debian environment of the Proxmox host machine, a dedicated Ubuntu 20.04 virtual machine was created. SecGen and its required dependencies were installed, and scenario execution was tested to confirm correct operation.

The third phase involved the adaptation of the Vagrant Proxmox provider. Because SecGen was originally designed to work with VirtualBox, it was necessary to modify and rebuild a forked version of the Proxmox provider. Several source code changes were implemented to correct API communication issues, SSL verification problems and permission errors.

The fourth phase consisted of integration testing between Ludus and SecGen. At this stage, the goal was to ensure that SecGen could successfully clone the templates created by Ludus and deploy vulnerable machines inside the Proxmox. Multiple tests were performed to validate template cloning, networking configuration and machine provisioning.

The fifth phase focused on the development of automation scripts. A custom Bash script was created to simplify the execution process, adjust configuration files automatically and reduce manual intervention when generating new vulnerable machines.

Finally, the last phase involved validation and testing of the complete solution. The

deployed cyber range was tested to verify template creation, range deployment, vulnerability provisioning. The results confirmed that the integration works as expected under the defined conditions.

This structured timeline ensured that each component was tested individually before full integration.

1.2 Document Structure

This report is organized into several chapters that present the concepts, development process, and results of the project. Chapter 2 introduces the fundamental concepts and the main technologies used in this work, namely Ludus, Proxmox, and SecGen. Chapter 3 describes the analysis and modelling approach adopted to design the proposed solution and integrate the different tools involved in the project.

Chapter 4 details the implementation phase, explaining the configuration procedures and the steps required to deploy the developed solution. Chapter 5 presents the testing process used to validate the system and discusses the results obtained during these experiments.

Finally, Chapter 6 summarizes the main conclusions of the project and outlines possible directions for future work. Appendix A contains the original project proposal, while the remaining appendices provide additional material, including relevant configuration files and modified source code used throughout the development of the project.

Chapter 2

Context and Tools

This chapter introduces the main concepts needed to understand the context of this project. To support the development of the proposed solution, it was necessary to study the concept of cyber ranges and analyse the main technologies and tools involved in this work, particularly SecGen and Ludus. This chapter therefore provides an overview of these concepts and technologies, which form the basis for the work presented in the following chapters.

A cyber range can be understood as a virtual environment designed to simulate real information technology infrastructures. Within this environment, it is possible to recreate networks, servers, applications and user activity, allowing cybersecurity techniques to be practiced in a controlled and isolated setting. This makes it possible to simulate attacks, test defensive strategies and analyse how systems behave under different security situations [2].

Cyber ranges are commonly used in universities, organizations and military training environments. They provide a safe space where students and professionals can experiment with cybersecurity techniques such as vulnerability analysis, penetration testing and incident response. Since the environment is isolated from real infrastructures, mistakes made during exercises do not affect production systems, which allows users to learn and experiment more freely.

In general, cyber ranges share several important characteristics. They operate in

isolation from real production systems, ensuring that exercises remain contained within the training environment. They also allow the creation of configurable and reproducible scenarios, meaning that the same environment can be recreated whenever needed. In addition, they support both attack and defence activities, while enabling monitoring and evaluation of the exercises carried out within the platform.

The project presented in this report follows this concept. Its main goal is to build a flexible and reproducible cyber range platform. In this context, SecGen is used to generate vulnerable systems and training scenarios, while Ludus is responsible for orchestrating and managing the infrastructure of the cyber range. By combining these tools, it becomes possible to create cybersecurity training environments that can be deployed, reproduced and adapted to different learning scenarios.

2.1 Technologies and Tools

This project is mainly practical and required the integration of several technologies. The most relevant tools are described below.

2.1.1 Virtualization Platforms

This section introduces the virtualization platforms used in this project. Virtualization plays a key role in building cyber range environments, as it makes it possible to create and manage multiple virtual machines and network components on the same physical infrastructure. The following subsections briefly present the platforms used and explain their role in supporting the implementation of the proposed solution.

Proxmox

Proxmox[3] is a virtualization platform that supports KVM virtual machines and LXC containers. It provides a web interface to manage virtual machines, storage, networking and snapshots.

In this project, Proxmox is used as the main virtualization host. It is responsible for creating, cloning and managing all virtual machines used in the cyber range. Through the use of resource pools, user groups, and access control lists (ACLs), it is possible to control which users can create, clone or manage virtual machines. Additionally, VLAN based network segmentation is used to isolate different ranges, ensuring that each group of users operates within its own secure and controlled environment.

KVM

Kernel based Virtual Machine (KVM)[4] is a virtualization technology built into the Linux kernel. It allows a Linux system to function as a hypervisor, meaning it can run multiple virtual machines on the same physical hardware.

KVM uses hardware virtualization extensions available in modern processors to provide efficient and secure isolation between virtual machines. Each virtual machine runs as a separate process in the host system and has its own virtual CPU, memory, storage and network interfaces.

In this project, KVM is the core virtualization technology used by Proxmox to execute virtual machines. All Ludus ranges and SecGen generated machines ultimately run on top of KVM.

QEMU

QEMU[5] is a machine emulator and virtualizer. It can emulate different hardware architectures and is often used together with KVM to provide full virtualization capabilities.

When used with KVM, QEMU handles hardware emulation, such as virtual disks, network cards and other virtual devices, while KVM provides hardware accelerated virtualization.

In this project, QEMU works together with KVM inside Proxmox. When a template is cloned or a new virtual machine is created by Ludus or SecGen, QEMU is responsible for emulating the virtual hardware components of that machine. The integration of KVM and QEMU ensures efficient execution of the cyber range infrastructure.

2.1.2 Tools

This section introduces the main tools used during the development of this project. These tools support the creation, configuration, and management of the cyber range environment. In particular, platforms such as Ludus, SecGen, and Vagrant play an important role in automating the deployment of vulnerable systems and orchestrating the infrastructure required for the training scenarios. The following subsections provide a brief overview of each tool and its role within the project.

Ludus

Ludus[6] is an open source cyber range platform built on top of Proxmox. It allows the creation of structured virtual environments through configuration files, allowing users to define machines, networks, and routing behavior in a declarative way.

One of the main features of Ludus is its ability to build and manage virtual machine templates automatically. These templates can be used as base images for deploying cyber ranges. Ludus supports multiple operating systems. The official documentation presents the predefined templates supported by Ludus and provides guidance on how users can create and integrate custom templates according to their specific requirements¹.

Ludus also supports the deployment of complete virtual environments that may include routers, VLAN segmentation, firewall configurations and secure remote access mechanisms. Through its integration with Proxmox, it provides structured orchestration of virtual machines and network isolation within cyber range infrastructures.

SecGen

SecGen[7] is a tool that automatically generates vulnerable machines for training and penetration testing. It installs vulnerable services and configurations on virtual machines based on predefined scenarios.

Originally, SecGen was designed to operate mainly with VirtualBox, using Vagrant as

¹<https://docs.ludus.cloud/docs/templates>

the virtualization management layer. To enable compatibility with Proxmox, an alternative Vagrant provider is required. In this case, integration with Proxmox is achieved through the use of a dedicated Vagrant(Proxmox) provider, available as an open source project².

Vagrant

Vagrant[8] is a tool used to automate the creation and management of virtual machines. SecGen uses Vagrant to deploy vulnerable machines. Since the original Proxmox provider was not fully compatible with the current environment, a forked version was used and modified to ensure proper communication with the Proxmox API.

SSH

Secure Shell (SSH)[9] is a network protocol used to establish secure remote connections between systems.

SSH is commonly used for remote system administration, file transfer and command execution. Authentication can be performed using passwords or, more securely, through public and private key pairs. Key based authentication allows only authorized users to access a system, reducing the risk of unauthorized access.

In automation and configuration management environments, SSH plays a central role by enabling secure communication between a control node and multiple managed machines.

Ansible

Ansible[10] is an automation and configuration management tool that operates without requiring agents to be installed on target machines. It relies on SSH to communicate securely with remote systems.

Ansible is written in Python and uses Python libraries internally to execute tasks and manage configurations.

²<https://github.com/cliffe/vagrant-proxmox>

Automation tasks in Ansible are defined using playbooks. A playbook is a structured file written in YAML (Yet Another Markup Language) format that describes a set of tasks to be executed on one or more hosts. These tasks may include installing packages, configuring services, modifying files, or managing system settings.

Ludus internally uses Ansible to configure virtual machine templates and deploy ranges. When templates are built, Ansible ensures that required packages, services and configurations are correctly applied. This approach allows reproducible and consistent environments.

The use of Ansible simplifies infrastructure management and reduces manual configuration errors, which is especially important in cyber range environments where multiple machines must be deployed in a predictable way.

Puppet

Puppet[11] is a configuration management tool used to automate the installation and configuration of software. It allows systems to be described in a declarative way, meaning that the desired state of a system is defined and then automatically enforced.

SecGen uses Puppet to provision vulnerable services inside the virtual machines it creates. When a scenario is selected, Puppet applies specific configurations that intentionally introduce vulnerabilities. This may include installing insecure services, weak credentials or outdated software versions.

The use of Puppet in SecGen ensures that vulnerability scenarios are reproducible and consistent. Each time a scenario is executed, the same vulnerable configuration can be deployed automatically, which is important for training and testing purposes.

2.1.3 Programming Languages

This section briefly presents the programming languages used during the development of the project. These languages play an important role in the configuration, automation and integration of the different components involved in the platform.

Bash

Bash[12] scripting was used to automate several steps of the integration process. A custom script was developed to prepare configuration files, adjust parameters and execute SecGen correctly using Ludus templates. This script ensures that vulnerable machines can be created without manual reconfiguration of files every time.

Ruby

Ruby[13] is a dynamic, object oriented programming language known for its simplicity and expressive syntax. It is widely used in automation tools and web based platforms.

Both SecGen and Vagrant are implemented using Ruby. As a result, their internal logic, plugins and configuration mechanisms are written in this language.

Python

Python[14] is a high level, general purpose programming language widely used for automation, scripting, and infrastructure management. Its clear syntax and extensive ecosystem of libraries make it well suited for system orchestration, configuration tasks, and integration with external tools.

2.1.4 Virtual Private Network (VPN) - Wireguard

WireGuard[15] is a modern Virtual Private Network (VPN) protocol designed to establish secure communication channels between devices.

In Ludus, WireGuard is a mandatory component for accessing deployed ranges. When a new range is created, Ludus relies on a dedicated virtual machine responsible for routing and network segmentation. This virtual machine, designated as the router, manages network traffic, VLAN segmentation and firewall rules.

If a router virtual machine does not already exist, Ludus automatically creates one during the deployment of the first range. This router is responsible for segmenting ranges using VLANs, where each VLAN corresponds to the user identifier associated with the

range. Communication between VLANs is controlled through firewall rules configured on this router, ensuring isolation between different environments.

During the installation process, Ludus installs and configures WireGuard so that it becomes the only authorized network path to access the virtual machines inside the ranges.

WireGuard ensures security through the use of modern cryptographic mechanisms based on public and private key exchange. Each peer authenticates using its own private key, and encrypted communication is established only between authorized endpoints. This key based authentication model, combined with strict routing and firewall control on the router virtual machine, guarantees that the cyber range remains isolated and securely accessible.

Chapter 3

Approach

This chapter presents a detailed explanation of the problem addressed in this project and the proposed solution. The main challenge identified is the need for a structured, scalable and reusable cyber range platform capable of generating controlled vulnerable environments for cybersecurity education.

In traditional laboratory environments, vulnerable machines are often configured manually. This process is time consuming, difficult to reproduce and hard to maintain consistently across different academic levels.

The objective of this work is to design a unified architecture that combines infrastructure orchestration with automated vulnerability generation inside a Proxmox based virtualization environment. The proposed solution must allow the creation of isolated ranges, secure remote access, and the generation of reusable vulnerable scenarios with different levels of complexity.

When applied in a production academic environment, this solution enables the creation of structured cybersecurity laboratories for CTeSP, Bachelor's and Master's programs. Simpler ranges can be deployed for introductory courses.

3.1 Proposed Solution Architecture

The proposed architecture follows a modular and layered approach that separates infrastructure management from vulnerability provisioning, while maintaining controlled integration between both components.

At the infrastructure level, Proxmox acts as the virtualization platform, managing virtual machines, storage, network bridges, cloning operations and resource allocation. On top of Proxmox, Ludus functions as the orchestration layer responsible for deploying cyber ranges.

Ludus manages the lifecycle of templates and ranges. It deploys isolated environments that include a dedicated router virtual machine. This router is responsible for VLAN segmentation, where each VLAN corresponds to a Ludus user identifier. Through firewall configuration on the router, communication between VLANs can be controlled or restricted. During installation, Ludus configures WireGuard, which becomes the only authorized network for accessing machines inside the ranges. This ensures that all deployed environments remain isolated and accessible only through secure tunnels.

At the vulnerability provisioning level, SecGen is used to generate intentionally vulnerable machines based on predefined scenarios. These scenarios are applied automatically through configuration management mechanisms. Vulnerable machines can be fully cloned and transformed into reusable templates. This improvement allows Ludus to deploy new ranges directly from previously generated vulnerable templates, avoiding repeated provisioning and increasing efficiency.

The integration between orchestration and vulnerability generation is achieved through a customized layer based on a modified Vagrant Proxmox provider and automation scripts. This layer ensures that vulnerable machines can be created, cloned and reused inside the Proxmox environment without manual reconfiguration.

From a modeling perspective, the system can be described as a controlled workflow. Templates are prepared and managed at the orchestration level. Vulnerability scenarios are generated at the provisioning level. Resulting vulnerable machines can be transformed

into reusable full clones. These clones can then serve as base templates for new range deployments.

The primary target users of the system are academic institutions and cybersecurity instructors who require controlled laboratory environments for practical teaching. Administrators are responsible for managing templates, deploying ranges and preparing scenarios. Students access the deployed ranges through WireGuard and interact with vulnerable machines in a safe and isolated manner.

3.2 Core Operational Workflow

The integrated system performs the following main operations:

1. Manage and prepare base templates at the orchestration level;
2. Generate vulnerable machines using automated scenarios;
3. Perform full clone operations to create reusable vulnerable templates;
4. Deploy ranges directly from previously prepared vulnerable templates;
5. Provide secure remote access through WireGuard.

This workflow enables dynamic generation of vulnerabilities and efficient reuse of prepared scenarios. The result is a scalable and reproducible cyber range platform capable of supporting structured cybersecurity education in a production academic environment.

Chapter 4

Implementation

This chapter provides a detailed explanation of the implementation of the methodology discussed in chapter 3. It explains the installation and configuration of the main components, the technical problems encountered, and the modifications required to achieve full integration between Ludus and SecGen.

4.1 Ludus Installation on Proxmox

Proxmox was installed on the host machine using the default installation procedure, following the official documentation¹. No custom modifications were applied during installation.

Once Proxmox was installed and ready, Ludus² was installed directly on the host machine. The installation process required the following initial packages:

Initial Package Installation

```
apt update && apt install curl sudo git ca-certificates python3-debian  
curl -s https://ludus.cloud/install | bash
```

After installation, Ludus returned:

¹<https://www.proxmox.com/en/proxmox-ve/get-started>

²<https://docs.ludus.cloud/docs/category/quick-start>

Ludus install completed successfully

Root API key: ROOT.*****

A new proxmox user was created:

LUDUS_API_KEY='ROOT*****'

ludus user add --name "John Doe" --userid JD --admin --url https://127.0.0.1:8081

User credentials were retrieved:

ludus user creds get

To allow VM creation during template build, ACL permissions were assigned:

pveum aclmod /storage/local-zfs -user john-doe@pam -role PVEAdmin

Critical Configuration Change

The default configuration file B.1.1:

/opt/ludus/config.yml

Required modification of the storage format:

proxmox_vm_storage_format: raw

Without this change, template builds failed because the default storage format was incompatible.

4.2 Template Build and Range Deployment

The build generates only the default templates provided by Ludus. If further templates are required for specific scenarios, they can be created and integrated according to the procedures described in the official documentation³

Templates were built using:

³<https://docs.ludus.cloud/docs/templates>

```
ludus templates build
ludus templates logs -f
```

Range configuration was exported and modified:

```
ludus range config get > ludus-range-config.yml
```

Ludus was then used to build and deploy a test range containing a single vulnerable machine and a default router. The router automatically provides gateway and NAT functionality. The range configuration defined B.1.2:

```
template: secgen-vuln
full_clone: true
vlan: 10
ram_gb: 4
cpus: 2
```

Range deployment:

```
ludus range config set -f ludus-range-config.yml
ludus range deploy
ludus range status
```

The deployed range automatically included a router responsible for:

- Gateway configuration
- NAT for internet access
- VLAN control

4.2.1 Wireguard

To get the WireGuard configuration file for your user:

```
ludus user wireguard | tee ludus.conf
```

4.3 SecGen Installation on Ubuntu 20.04 VM

Initially, an attempt was made to install SecGen directly on the Proxmox host (Debian). However, compatibility issues were encountered. SecGen was not fully compatible with the Debian environment. To solve this issue, a dedicated Ubuntu Server 20.04 virtual machine was created inside Proxmox. SecGen⁴ was installed inside this VM following the official repository instructions.

Vagrant Installation

```
wget https://releases.hashicorp.com/vagrant/2.2.9/vagrant_2.2.9_x86_64.deb
sudo apt install ./vagrant_2.2.9_x86_64.deb
```

Required dependencies:

```
sudo apt-get install ruby-dev zlib1g-dev liblzma-dev \
build-essential patch virtualbox ruby-bundler \
imagemagick libmagickwand-dev exiftool \
libpq-dev libcurl4-openssl-dev libxml2-dev \
graphviz graphviz-dev libpcap0.8-dev git
```

Additional required package:

```
sudo apt install libsqlite3-dev
```

SecGen repository:

```
git clone https://github.com/cliffe/SecGen.git
cd SecGen
bundle update --bundler
bundle update
```

Validation:

```
ruby secgen.rb list-scenarios
```

⁴<https://github.com/cliffe/SecGen>

4.4 Adapting Vagrant Proxmox Provider

SecGen was originally designed to work with VirtualBox through Vagrant. In order to use Proxmox as the target virtualization platform, it was necessary to adapt the Vagrant Proxmox provider. A forked version of the provider was cloned and manually patched. The gem specification file was modified to relax the Ruby version requirement:

Clone and Build Provider

```
git clone https://github.com/cliffe/vagrant-proxmox.git
cd vagrant-proxmox
```

Gem specification modification B.2.3:

```
spec.required_ruby_version = '>= 2.6'
```

This change allowed the plugin to be built using the embedded Ruby version provided by Vagrant.

Build gem:

```
/opt/vagrant/embedded/bin/gem build vagrant-proxmox.gemspec
vagrant plugin expunge --force
vagrant plugin install ./vagrant-proxmox-*.gem
```

Dummy box (required by plugin):

```
git clone https://github.com/telcat/vagrant-proxmox.git
vagrant box add dummy telcat-vagrant-proxmox/dummy_box/dummy.box
```

4.5 Critical Source Code Modifications

4.5.1 Fix 1: API Endpoint Handling

One critical issue was related to the Proxmox API endpoint. The provider did not correctly handle endpoints without an explicit scheme.

File B.2.4:

`connection.rb`

Original code:

```
@api_url = api_url
```

Modified:

```
ep = api_url.to_s.strip
ep = "https://#{ep}" unless ep.start_with?("http://") || ep.start_with?("https://")
@api_url = ep
```

Without this fix, the provider attempted to connect through port 80, resulting in connection failures.

4.5.2 Fix 2: SSL Verification

Another issue was related to SSL certificate verification. Since the Proxmox server was using a self signed certificate, API calls were failing. To solve this, SSL verification was explicitly disabled in the provider.

Added:

```
require 'openssl'
```

Modified all RestClient calls to:

```
response = RestClient::Request.execute(
  method: :get,
  url: "#{api_url}#{path}",
  cookies: { PVEAuthCookie: ticket },
  verify_ssl: OpenSSL::SSL::VERIFY_NONE
)
```

This modification was applied to GET, POST and DELETE methods inside the connection file.

4.5.3 Fix 3: Force POST

Original headers:

```
def headers
  ticket.nil? ? {} : { CSRFPreventionToken: csrf_token,
    cookies: { PVEAuthCookie: ticket } }
end
```

Modified:

```
def headers
  base = { content_type: 'application/x-www-form-urlencoded' }
  ticket.nil? ? base :
  base.merge(CSRFPreventionToken: csrf_token,
    cookies: { PVEAuthCookie: ticket })
end
```

Prevent 400 Bad Request errors when sending POST.

4.5.4 Fix 4: Skip Forwarded Port Logic

Another error occurred due to forwarded port configuration logic in the provider. Ludus templates already define network settings, and forwarded ports were not required.

File B.2.5:

config_clone.rb

To bypass this issue, the following block was inserted in:

```
if config.disable_adjust_forwarded_port
  env[:ui].detail "Skipping node ip / forwarded port adjustments"
  return @app.call(env)
end
```

This prevented unnecessary network adjustments that were causing 400 Bad Request errors.

4.5.5 Fix 5: ACL Permissions

When cloning templates, the provider generated HTTP 403 errors. The issue was related to insufficient permissions in Proxmox.

ACL rules were adjusted using:

```
pveum aclmod /pool/SHARED -user john-doe@pam -role PVEAdmin
pveum aclmod /vms -user john-doe@pam -role PVEVMAdmin
```

4.5.6 Fix 6: Full Clone

For the SecGen template to be removable at a later stage, the vulnerable machine must be created as a full clone and then manually converted into a template. Once Ludus deploys a range from this template, the template can be deleted by the administrator without impacting the deployed range, as the range operates independently after creation.

File B.2.6:

clone_vm.rb

Modified:

```
def create_params_qemu(config, env, vm_id, template_vm_id)
  { vmid: template_vm_id,
    newid: vm_id,
    name: env[:machine].config.vm.hostname || env[:machine].name.to_s,
    description: "#{config.vm_name_prefix}#{env[:machine].name}",
    full: 1 }
```

4.6 Integration Script Development

To automate the integration between Ludus and SecGen, a custom Bash script B.2.2 was developed.

`script.sh`

The script performs the following tasks:

- Verifies if the required Ludus template exists;
- Clones the template;
- Replaces the default SecGen Vagrantfile with a custom template;
- Injects correct Proxmox parameters;
- Executes SecGen to provision vulnerabilities.

A custom template file was created B.2.1:

`~/SecGen/Vagrantfile.proxmox.ludus.template`

This file defines the Proxmox provider configuration and ensures compatibility with Ludus templates.

The script dynamically adjusts parameters such as:

- Template name;
- VM ID range;
- VM name prefix.

This prevents manual configuration before each execution.

The system successfully integrates Ludus and SecGen, allowing the creation of vulnerable machines inside controlled cyber ranges managed by Proxmox.

Chapter 5

Tests

This chapter presents the tests performed to validate the developed solution. The objective was to verify that the integration between Ludus and SecGen works correctly, and that vulnerable machines can be generated, converted into templates and deployed inside isolated ranges.

Each test is described with its objective, expected result and obtained result.

5.1 Ludus Template Preparation

Description

The first step consisted of building the default templates using Ludus. These templates serve as the base infrastructure for vulnerability generation and range deployment.

Expected Result

The templates should be successfully created and registered in Proxmox, making them available for cloning operations.

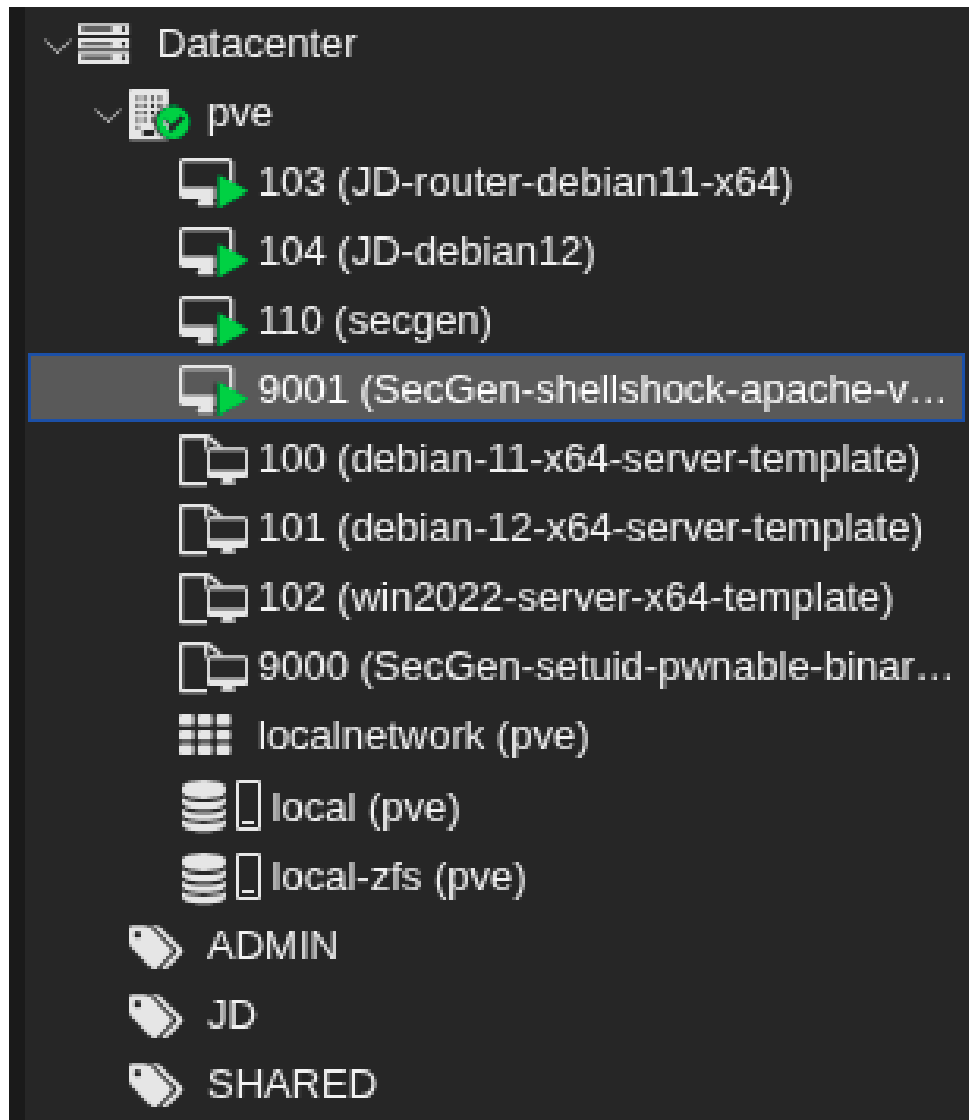


Figure 5.1: Overview of the Proxmox node (pve).

Obtained Result

The templates were successfully created and listed in Proxmox. This confirmed that the infrastructure base was correctly prepared and ready for integration with SecGen.

5.2 Vulnerable Machine Generation with SecGen

Description

After preparing the templates, SecGen was executed to generate vulnerable virtual machines. The vulnerable machines were created using the Ludus Debian 12 template as a base image and generated as full clones.

Expected Result

SecGen should:

- Clone the Ludus Debian 12 template as a full clone.
- Apply a predefined vulnerability scenario.
- Produce a functional vulnerable virtual machine.

Obtained Result

Several SecGen scenarios were tested. However, it was observed that some scenarios are no longer fully compatible with recent Debian versions, as they were developed some time ago. In those cases, provisioning errors occurred during the vulnerability injection process.

The scenarios that completed successfully were able to clone the Debian 12 template using the full clone method and correctly apply the intended vulnerabilities. After creation, the virtual machine was analyzed to verify that the vulnerabilities had been properly implemented.

Only the machines where vulnerability injection was successfully completed were manually converted into templates for later deployment in Ludus.

5.3 Conversion to Template

Description

Once the vulnerable machine was validated, it was manually converted into a Proxmox template. This step allows the vulnerable configuration to be reused for future deployments.

Expected Result

The vulnerable machine should be successfully converted into a reusable template without losing its configuration.

Obtained Result

The vulnerable virtual machine was successfully converted into a template. Because it was created as a full clone, it became independent from the original template structure. This allows the administrator to delete the vulnerable template later if necessary, without affecting any deployed ranges.

5.4 Range Deployment with Vulnerable Machines

Description

Using Ludus, a new range was deployed based on the previously created vulnerable template. The range was configured inside an isolated network environment.

During deployment, Ludus automatically created a router virtual machine based on Debian 11. This router is responsible for DHCP services, gateway configuration and VPN

access through WireGuard.

Expected Result

The deployment should:

- Create an isolated range;
- Deploy the vulnerable machine from the custom template;
- Automatically create the router virtual machine;
- Configure DHCP and VPN access.

Obtained Result

The range was successfully deployed. The vulnerable machine was created from the generated template, and the router virtual machine was automatically provisioned by Ludus using Debian 11.

The router provided DHCP services, NAT configuration and controlled access via WireGuard. The range operated in an isolated environment, ensuring that the vulnerable machine was not directly exposed to external networks.

User ID	Range Network	Last Deployment	Number of VMs	Deployment Status	Testing Enabled
JD	10.2.0.0/16	2026-02-26 14:37	2	SUCCESS	FALSE

Tabela 5.1: Ludus range information.

5.5 Discussion

The performed tests demonstrate that the integration between Ludus and SecGen is functional and stable under the tested conditions.

The following conclusions can be drawn:

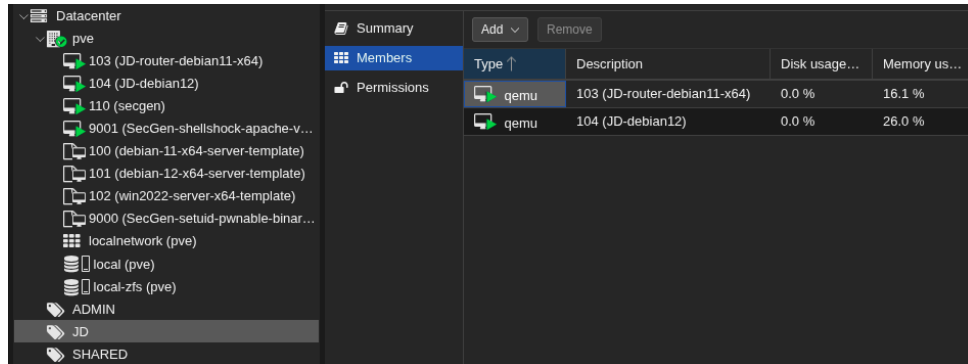


Figura 5.2: Range of user JD.

- Vulnerable machines can be generated using SecGen from Ludus templates;
- Full clone creation enables template reuse and safe deletion after deployment;
- Ludus successfully deploys isolated ranges including router, DHCP and VPN access;
- Infrastructure orchestration and vulnerability generation can operate together in a Proxmox environment.

The results confirm that the proposed solution achieves its main objective, enabling the creation of structured, reusable and isolated vulnerable environments for academic cybersecurity training.

Chapter 6

Conclusions

The main objective of this project was to design and implement an integrated cyber range platform based on Proxmox, combining infrastructure orchestration with automated vulnerability generation. The intention was to create a structured and reusable environment suitable for cybersecurity education at different academic levels.

The integration between Ludus and SecGen was successfully achieved. Ludus manages templates, deploys isolated ranges, configures network segmentation through virtual local area networks, and provides secure access through WireGuard. SecGen was adapted to operate within the Proxmox environment, allowing vulnerable virtual machines to be generated from existing templates.

During testing, vulnerable machines were created using the Debian 12 template provided by Ludus. These machines were generated as full clones and, after validation, were manually converted into templates. Once converted, they were used by Ludus to deploy new isolated ranges. After deployment, the template could be removed without affecting the operational range, since the deployed environment no longer depends on the original template. This approach ensures flexibility and efficient resource management.

Some limitations were identified during testing. Certain SecGen scenarios are no longer fully compatible with recent operating system versions. For this reason, only the scenarios that successfully completed vulnerability injection were converted into templates and reused.

In conclusion, the project resulted in a stable and operational cyber range solution capable of supporting practical cybersecurity training. The platform provides a structured and controlled environment where vulnerabilities can be explored safely without compromising institutional infrastructure.

Future Work

Although the core objectives of the project were achieved, several improvements can be implemented to increase automation and robustness.

One important enhancement would be the development of a compatibility verification mechanism within the SecGen execution process. Before executing a scenario, the system could automatically verify whether the selected scenario is compatible with the current template version. This would prevent execution errors and improve overall reliability.

Another significant improvement would be the automation of the post provisioning phase. After successfully generating a vulnerable virtual machine, the system could automatically perform a controlled shutdown and convert the machine into a template. This would eliminate the need for manual intervention and transform the current semi automated workflow into a fully automated process.

Bibliography

- [1] OpenAI. *ChatGPT*. <https://chat.openai.com/>.
- [2] NIST. *THE CYBER RANGE A GUIDE*. Community Coordinating Council, 2023.
- [3] Proxmox Server Solutions GmbH. *Proxmox Virtual Environment*. <https://www.proxmox.com/en/proxmox-ve>.
- [4] Linux Kernel Organization. *Kernel-based Virtual Machine (KVM)*. <https://www.linux-kvm.org/>.
- [5] Fabrice Bellard. *QEMU: Open Source Machine Emulator*. <https://www.qemu.org/>.
- [6] Ludus Project. *Ludus: Open-Source Cyber Range Platform*. <https://docs.ludus.cloud/>.
- [7] Neil Cliffe. *SecGen: Automated Generation of Vulnerable Machines*. <https://github.com/cliffe/SecGen>.
- [8] HashiCorp. *Vagrant by HashiCorp*. <https://www.vagrantup.com/>.
- [9] Tatu Ylonen e Chris Lonvick. *The Secure Shell (SSH) Protocol Architecture*. <https://www.rfc-editor.org/rfc/rfc4251>.
- [10] Red Hat. *Ansible Automation Platform*. <https://www.ansible.com/>.
- [11] Puppet Inc. *Puppet Configuration Management Tool*. <https://github.com/davidmarquis/vagrant-puppet>.
- [12] Free Software Foundation. *GNU Bash Reference Manual*. <https://www.gnu.org/software/bash/>.

- [13] Yukihiro Matsumoto. *Ruby Programming Language*. <https://www.ruby-lang.org/>.
- [14] Python Software Foundation. *Python Programming Language*. <https://www.python.org/>.
- [15] Jason A. Donenfeld. *WireGuard*. <https://www.wireguard.com/>.

Appendix A

Original Project Proposal

Automatização de Cenários “cyber range”

☐	Inteligência artificial	☐	Multimédia/Realidade aumentada/...
☐	Aplicações móveis	☐	Redes e gestão de sistemas
☐	Plataformas web	☒	Cibersegurança

* - A especificar pelo proponente

Orientador: Docente, Tiago Pedrosa

Coorientador: Docente, Jorge Loureiro

1 Objetivo

O objetivo é desenvolver uma solução com diversos cenários concorrentes de automatização "cyber range" para formação e treino em cibersegurança e análise de vulnerabilidades nos cenários gerados.

2 Detalhes

A primeira fase consiste na análise do estado de arte, estudo e pesquisa de metodologias e ferramentas open-source, permitindo identificar soluções existentes, limitações e oportunidades de melhoria.

Com base no estudo realizado será elaborada uma proposta da solução, definindo os tipos de cenários a automatizar, os requisitos técnicos e os mecanismos de integração entre as ferramentas envolvidas.

Segue-se a fase de desenvolvimento e implementação, onde serão criados os primeiros cenários automatizados. Estes cenários serão projetados para simular ambientes realistas, contendo vulnerabilidades e desafios.

Posteriormente, serão realizados testes, com o objetivo de avaliar a eficácia dos cenários e identificar eventuais melhorias.

Ao longo de todo o projeto, será feita a devida documentação técnica, incluindo descrições dos cenários, configurações utilizadas, problemas encontrados e soluções adotadas. Por fim, será elaborado o relatório final, reunindo todo o trabalho desenvolvido, os resultados obtidos e sugestões para trabalho futuro.

3 Metodologia de trabalho

- Análise do estado de arte,
- Estudo e pesquisa da solução,
- Proposta da solução,
- Desenvolvimento e implementação da solução,
- Testes e melhorias,
- Documentação,
- Escrita do relatório,

Dimensão da equipa:	1
Recursos necessários:	Estação de trabalho com capacidade de virtualização do cenário e desenvolvimento. Um servidor com capacidade de virtualização para testes.

Appendix B

Other Appendices

B.1 Ludus

B.1.1 config.yml

/opt/ludus/config.yml

```
proxmox_node: pve
proxmox_interface: vmbr0
proxmox_invalid_cert: true
proxmox_url: https://127.0.0.1:8006
proxmox_hostname: pve
proxmox_local_ip: 10.1.1.184
proxmox_public_ip: 10.1.1.184
proxmox_gateway: 10.1.1.1
proxmox_netmask: 255.255.255.0
proxmox_vm_storage_pool: local-zfs
proxmox_vm_storage_format: raw
proxmox_iso_storage_pool: local
ludus_nat_interface: vmbr1000
prevent_user_ansible_add: false
license_key: community
expose_admin_port: false
reserved_range_numbers: []
```

B.1.2 ludus-range-config.yml

```
# yml-language-server: $schema=https://docs.ludus.cloud/schemas/range-config.json

ludus:
  - vm_name: "{{ range_id }}-debian12"
    hostname: "{{ range_id }}-debian12"
    template: secgen-vuln
    full_clone: true
    vlan: 10
    ip_last_octet: 11
    ram_gb: 4
    cpus: 2
    linux: true
    testing:
      snapshot: false
      block_internet: false

network:
  external_default: ACCEPT
  inter_vlan_default: REJECT
  rules: []
```

B.2 Secgen

B.2.1 Vagrantfile.proxmox.ludus.template

```
VAGRANTFILE_API_VERSION = "2"

Vagrant.configure(VAGRANTFILE_API_VERSION) do |config|
  # Plugin precisa sempre disto
```

```
config.vm.box = "dummy"

config.vm.synced_folder ".", "/vagrant", disabled: true

# SSH do template Ludus (debian:debian)
config.ssh.username = "debian"
config.ssh.password = "debian"
config.ssh.insert_key = false
config.ssh.verify_host_key = false

config.vm.provider :proxmox do |proxmox|
  proxmox.endpoint = "IP:8006/api2/json"
  proxmox.verify_ssl = false

  proxmox.user_name = "john-doe@pam"
  proxmox.password = "*****"

  proxmox.pool = "JD"
  proxmox.selected_node = "pve"

  proxmox.vm_id_range = 9000..9100
  proxmox.vm_name_prefix = "secgen-"
  proxmox.vm_type = :qemu

  proxmox.qemu_template = "debian-12-x64-server-template"

  proxmox.qemu_os = :l26
  proxmox.qemu_disk_size = "100G"

  proxmox.disable_adjust_forwarded_port = true
end
end
```


B.2.2 script.sh

```
#!/usr/bin/env bash
set -euo pipefail

SECGEN_DIR="$HOME/SecGen"
PROJ_ROOT="$SECGEN_DIR/projects"

# --- Proxmox/Ludus ---
PVE_API="IP:8006/api2/json"
PVE_NODE="pve"
PVE_USER="john-doe@pam"
PVE_PASS="*****"
PVE_POOL="JD"
PVE_TEMPLATE_NAME="debian-12-x64-server-template"

VM_SSH_USER="debian"
VM_SSH_PASS="debian"

cd "$SECGEN_DIR"

# 0) Escolher scenario random
SCENARIO="$(find escenarios/examples -maxdepth 3 -name '*.xml' | shuf -n 1)"
echo "[+] Scenario: $SCENARIO"

# 1) Gerar o project
ruby secgen.rb --scenario "$SCENARIO" build-project

# 2) Detetar o project criado
LATEST_PROJ="$(find "$PROJ_ROOT" -maxdepth 1 -type d -name 'SecGen*' -printf '%T@ %p\n'
    ' | sort -nr | head -n 1 | cut -d' ' ' -f2-)"
echo "[+] Project: $LATEST_PROJ"

VF="$LATEST_PROJ/Vagrantfile"
cp "$VF" "$VF.secgen.orig"
```

```

# 3) Remover blocos do provider
,

BEGIN{skip=0}
/^[[:space:]]*[A-Za-z0-9_]+\.\vm\.\provider[[:space:]]+:\virtualbox[[:space:]]+do[[:
    space:]]+\|[A-Za-z0-9_]+\|[[:space:]]*$/ {skip=1; next}
skip==1 && /^[[:space:]]*end[[:space:]]*$/ {skip=0; next}
skip==1 {next}
{print}
' "$VF" > "$VF.tmp" && mv "$VF.tmp" "$VF"

# 4) Injetar template Proxmox
INJECT_FILE="$LATEST_PROJ/.proxmox_inject.vf"
cat > "$INJECT_FILE" <<EOF
# --- Proxmox/Ludus (injetado) ---
config.vm.box = "dummy"
config.vm.synced_folder ".", "/vagrant", disabled: true

config.ssh.username = "${VM_SSH_USER}"
config.ssh.password = "${VM_SSH_PASS}"
config.ssh.insert_key = true
config.ssh.verify_host_key = false

config.vm.provider :proxmox do |proxmox|
    proxmox.endpoint = "${PVE_API}"
    proxmox.verify_ssl = false

    proxmox.user_name = "${PVE_USER}"
    proxmox.password = "${PVE_PASS}"

    proxmox.pool = "${PVE_POOL}"
    proxmox.selected_node = "${PVE_NODE}"

    proxmox.vm_id_range = 9000..9100
    proxmox.vm_name_prefix = "secgen-"

```

```

proxmox.vm_type = :qemu

proxmox.qemu_template = "${PVE_TEMPLATE_NAME}"
proxmox.qemu_os = :l26
proxmox.qemu_disk_size = "200G"

proxmox.qemu_storage = "local-zfs"
proxmox.qemu_bridge = "vmbr1000"
proxmox.qemu_nic_model = "virtio"

proxmox.disable_adjust_forwarded_port = true
end

# Garantir dependencias para rsync/puppet dentro da VM
config.vm.provision "shell", inline: <<'SHELL'
set -eux
export DEBIAN_FRONTEND=noninteractive
apt-get update -y
apt-get install -y rsync puppet
puppet --version
SHELL

EOF

LINE="$(grep -n -m1 'Vagrant\.configure' "$VF" | cut -d: -f1 || true)"
if [[ -z "${LINE}" ]]; then
    echo "[!] ERRO: nao foi encontrado 'Vagrant.configure' em $VF"
    exit 1
fi

sed -i "${LINE}r $INJECT_FILE" "$VF"
rm -f "$INJECT_FILE"

# 5) Trocar redes estaticas por DHCP

```

```

sed -i -E 's/(\.vm\.network[[:space:]]+:private_network),[[:space:]]+ip:[[:space:]]+:[[:space:]]+:[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+"/\1, type: "dhcp", auto_config: false/g' "$VF"

echo "[+] Vagrantfile patchado para Proxmox."

# 6) Validar que o provider proxmox existe
echo "[+] Confirmar provider proxmox no Vagrantfile:"
grep -n 'config\.vm\.provider[[:space:]]*::proxmox' "$VF" >/dev/null || { echo "[!] Falhou: no existe provider proxmox no Vagrantfile"; exit 1; }
grep -n 'proxmox\.endpoint' "$VF" >/dev/null || { echo "[!] Falhou: no existe proxmox.endpoint"; exit 1; }
grep -n 'proxmox\.user_name' "$VF" >/dev/null || { echo "[!] Falhou: no existe proxmox.user_name"; exit 1; }
grep -n 'proxmox\.password' "$VF" >/dev/null || { echo "[!] Falhou: no existe proxmox.password"; exit 1; }
grep -n 'proxmox\.vm_type' "$VF" >/dev/null || { echo "[!] Falhou: no existe proxmox.vm_type"; exit 1; }

# 7) Subir VMs
cd "$LATEST_PROJ"
vagrant validate
vagrant up --provider=proxmox

echo "[+] DONE: $LATEST_PROJ"

```

B.2.3 vagrant-proxmox.gemspec

```

$LOAD_PATH.unshift File.expand_path '../lib', __FILE__
require 'vagrant-proxmox/version'

Gem::Specification.new do |spec|
  spec.name = 'vagrant-proxmox'
  spec.version = VagrantPlugins::Proxmox::VERSION
  spec.platform = Gem::Platform::RUBY

```

```

spec.license = 'MIT'
spec.authors = ['Dirk Grappendorf', 'Tim Voelpel', 'Sebastian Bremicker',
               'Sebastian Lehn']
spec.email = ['dirk.grappendorf@telcat.de', 'tim.voelpel@telcat.de',
              'sebastian.bremicker@telcat.de', 'lehn@etracker.com']
spec.homepage = 'https://github.com/telcat/vagrant-proxmox'
spec.summary = 'Enables Vagrant to manage virtual machines on a Proxmox server.'
spec.description = 'Enables Vagrant to manage virtual machines on a Proxmox\'
                  \' server. \'
spec.required_ruby_version = '>= 2.6'
spec.required_rubygems_version = '>= 1.3.6'

spec.add_dependency 'rest-client', '>= 2.0.0', '< 3.0'
spec.add_runtime_dependency 'retryable', '~> 1.3', '>= 1.3.3'
spec.add_runtime_dependency 'activesupport', '~> 7.0.4'
spec.add_development_dependency 'rake', '13.0.6'
spec.add_development_dependency 'rspec', '~> 3.0'
spec.add_development_dependency 'simplecov', '~> 0.9.0'
spec.add_development_dependency 'simplecov-rcov', '~> 0.2.3'
spec.add_development_dependency 'geminabox', '~> 0.11.1'
spec.add_development_dependency 'guard-rspec', '4.2.10'
spec.add_development_dependency 'libnotify', '~> 0.8.3'
spec.add_development_dependency 'timecop', '~>0.7.1'
spec.add_development_dependency 'cucumber', '~> 1.3', '>= 1.3.15'
spec.add_development_dependency 'webmock', '~> 1.18', '>= 1.18.0'
spec.add_development_dependency 'awesome_print', '~> 1.2', '>= 1.2.0'

spec.files = Dir.glob('lib/**/*.rb') + Dir.glob('locales/**/*.yaml')
spec.test_files = Dir.glob 'spec/**/*.rb'
spec.require_paths = %w(lib)
end

```

B.2.4 connection.rb

`/.vagrant.d/gems/3.3.8/gems/vagrant-proxmox-0.3.0/lib/vagrant-proxmox/proxmox/connection.rb`

```
require 'vagrant-proxmox/proxmox/errors'
require 'rest-client'
require 'retryable'
require 'required_parameters'
require 'json'
require 'fileutils'
require 'openssl'
# require 'irb'

module VagrantPlugins
  module Proxmox
    class Connection
      include RequiredParameters

      attr_reader :api_url
      attr_reader :ticket
      attr_reader :csrf_token
      attr_accessor :vm_id_range
      attr_accessor :task_timeout
      attr_accessor :task_status_check_interval
      attr_accessor :imgcopy_timeout
      attr_accessor :vm_info_cache

      def initialize(api_url, opts = {})
        ep = api_url.to_s.strip
        # Ensure scheme is present; Proxmox API is normally HTTPS on 8006
        ep = "https://#{ep}" unless ep.start_with?("http://") || ep.start_with?("https://")
        @api_url = ep
        @vm_id_range = opts[:vm_id_range] || (900..999)
        @task_timeout = opts[:task_timeout] || 600
      end
    end
  end
end
```

```

    @task_status_check_interval = opts[:task_status_check_interval] || 2
    @imgcopy_timeout = opts[:imgcopy_timeout] || 1200
    @vm_info_cache = {}
end

def login(username: required('username'), password: required('password'))
  response = post '/access/ticket', username: username, password: password
  @ticket = response[:data][:ticket]
  @csrf_token = response[:data][:CSRFPreventionToken]
rescue ApiError::ServerError
  raise ApiError::InvalidCredentials
rescue => x
  raise ApiError::ConnectionError, x.message
end

def get_node_list
  nodelist = get '/nodes'
  nodelist[:data].map { |n| n[:node] }
end

def get_vm_state(vm_id, node)
  begin
    response = get "/nodes/#{node}/qemu/#{vm_id}/status/current"
    states = { 'running' => :running,
               'stopped' => :stopped }
    states[response[:data][:status]]
  rescue ApiError::ServerError
    :not_created
  end
end

def wait_for_completion(task_response: required('task_response'), timeout_message
  : required('timeout_message'))

  task_upid = task_response[:data]

```

```

    timeout = task_timeout
    task_type = /UPID:.*?:.*?:.*?:.*?:(.*)??:.*?:.*?:/.match(task_upid)[1]
    timeout = imgcopy_timeout if task_type == 'imgcopy' || task_type == 'qmclone'
    begin
      retryable(on: VagrantPlugins::Proxmox::ProxmoxTaskNotFinished,
                tries: timeout / task_status_check_interval + 1,
                sleep: task_status_check_interval) do
        exit_status = get_task_exitstatus task_upid
        exit_status.nil? ? raise(VagrantPlugins::Proxmox::ProxmoxTaskNotFinished) :
          exit_status
      end
    rescue VagrantPlugins::Proxmox::ProxmoxTaskNotFinished
      raise VagrantPlugins::Proxmox::Errors::Timeout, timeout_message
    end
  end

  def delete_vm(vm_id, node)
    response = delete "/nodes/#{node}/qemu/#{vm_id}"
    wait_for_completion task_response: response, timeout_message: 'vagrant_proxmox.
      errors.destroy_vm_timeout'
  end

  def qemu_agent_get_ip(vm_id, node)
    begin
      # binding.irb
      response = get "/nodes/#{node}/qemu/#{vm_id}/agent/network-get-interfaces"
    rescue ApiError::ServerError
      return nil
    rescue RestClient::InternalServerError
      return nil
    rescue VagrantPlugins::Proxmox::ApiError::ServerError
      return nil
    end
    # select the first nic (0 = lo, 1 = first nic)
    response.dig(:data, :result).each do |nic|

```



```

    nic.dig(:"ip-addresses").each do |ip_addresses_block|
      # find an IPv4 address and return it
      if ip_addresses_block.dig(:"ip-address-type") == "ipv4"
        ip = ip_addresses_block.dig(:"ip-address")
        return ip if ip != "127.0.0.1"
      end
    end
  end
end

nil
end

def create_vm(node: required('node'), vm_type: required('node'), params: required(
  'params'))
  response = post "/nodes/#{node}/#{vm_type}", params
  wait_for_completion task_response: response, timeout_message: 'vagrant_proxmox.
    errors.create_vm_timeout'
end

def clone_vm(node: required('node'), vm_type: required('node'), params: required(
  'params'))
  # We get locking errors from Proxmox when provisioning multiple clones at once
  lock_file_path = '/tmp/clone_vm_lock'
  # Get the exclusive lock
  File.open(lock_file_path, File::CREAT) do |file|
    acquired_lock = file.flock(File::LOCK_EX | File::LOCK_NB) # Try to acquire an
      exclusive lock without blocking
    # don't continue until we have the lock
    lock_timeout = 1200 # after 30 mins sod it off and delete the lock
    sleep_interval = 1
    unless acquired_lock
      start_time = Time.now
      loop do
        break if file.flock(File::LOCK_EX | File::LOCK_NB) # Break if the lock is
          acquired
        elapsed_time = Time.now - start_time

```

```

    if elapsed_time % 5 == 0
      puts "Waiting for lock: #{elapsed_time.round} seconds waiting..."
    end

    # if it takes too long, then delete the lock and try again
    if elapsed_time >= lock_timeout
      puts "Timeout: Unable to acquire the lock within #{lock_timeout} seconds
        . Deleting lock file."
      File.delete(lock_file_path) if File.exist?(lock_file_path)
    end

    sleep sleep_interval
  end
end

# Critical section
vm_id = params[:vmid]
params.delete(:vmid)
params.delete(:ostype)
params.delete(:ide2)
params.delete(:sata0)
params.delete(:sockets)
params.delete(:cores)
params.delete(:description)
params.delete(:memory)
params.delete(:net0)
retries = 30
while retries > 0
  response = post "/nodes/#{node}/#{vm_type}/#{vm_id}/clone", params
  wait_response = wait_for_completion task_response: response,
    timeout_message: 'vagrant_proxmox.errors.create_vm_timeout'
  if wait_response == "OK"
    break
  else
    puts "Failed to clone VM. Retrying..."
  end
end

```

```

        retries -= 1
        sleep rand(20..40) # random sleep time between 5 and 10 seconds
    end
end
if retries == 0
    puts "Failed to clone VM after multiple retries."
end

# Release the lock
file.flock(File::LOCK_UN)

wait_response
end
end

def config_clone(node: required('node'), vm_type: required('node'), params:
    required('params'))
    vm_id = params[:vmid]
    params.delete(:vmid)
    response = post "/nodes/#{node}/#{vm_type}/#{vm_id}/config", params
    wait_for_completion task_response: response, timeout_message: 'vagrant_proxmox.
        errors.create_vm_timeout'
end

def get_vm_config(node: required('node'), vm_id: required('node'), vm_type:
    required('node'))
    response = get "/nodes/#{node}/#{vm_type}/#{vm_id}/config"
    response = response[:data]
    response.empty? ? raise(VagrantPlugins::Proxmox::Errors::VMConfigError) :
        response
end

def start_vm(vm_id, node)
    response = post "/nodes/#{node}/qemu/#{vm_id}/status/start", nil

```

```

    wait_for_completion task_response: response, timeout_message: 'vagrant_proxmox.
        errors.start_vm_timeout'
end

def stop_vm(vm_id, node)
    response = post "/nodes/#{node}/qemu/#{vm_id}/status/stop", nil
    wait_for_completion task_response: response, timeout_message: 'vagrant_proxmox.
        errors.stop_vm_timeout'
end

def shutdown_vm(vm_id, node)
    response = post "/nodes/#{node}/qemu/#{vm_id}/status/shutdown", nil
    wait_for_completion task_response: response, timeout_message: 'vagrant_proxmox.
        errors.shutdown_vm_timeout'
end

# when given a large range we can optimise by simply choosing an id randomly
# otherwise we list all the possible ids and compare to what's available
def get_free_vm_id
    if vm_id_range.size > 1000
        # just randomly choose from within the large range
        rand(vm_id_range)
    else
        # to avoid collisions in multi-vm setups
        sleep (rand(1..3) + 0.1 * rand(0..9))
        response = get '/cluster/resources?type=vm'
        allowed_vm_ids = vm_id_range.to_set
        used_vm_ids = response[:data].map { |vm| vm[:vmid] }
        free_vm_ids = (allowed_vm_ids - used_vm_ids).sort
        free_vm_ids.empty? ? raise(VagrantPlugins::Proxmox::Errors::NoVmIdAvailable)
            : free_vm_ids.first
    end
end

def get_qemu_template_id(template)

```

```

# Check if the cache file exists
cache_file_path = '/tmp/template_ids_cache.json'
template_cache = {}

# Attempt to acquire an exclusive lock on the cache file
File.open(cache_file_path, File::RDWR | File::CREAT, 0644) do |file|
  file.flock(File::LOCK_EX)

  if File.exist?(cache_file_path)
    content = file.read
    template_cache = JSON.parse(content) unless content.empty?
    puts "Cache loaded from #{cache_file_path}: #{template_cache}"
    return template_cache[template] if template_cache&.key?(template)
  else
    puts "Cache file #{cache_file_path} not found."
  end

  # If the template is not found in the cache, fetch from the API
  response = get '/cluster/resources?type=vm'

  # Update cache and write it back to file
  found_ids = response[:data].select { |vm| vm[:type] == 'qemu' }.select { |vm|
    vm[:template] == 1 }.select { |vm| vm[:name] == template }.map { |vm| vm
    [:vmid] }
  puts "Found IDs from API: #{found_ids}"

  if found_ids.empty?
    puts "No template ID found for '#{template}'."
    raise(VagrantPlugins::Proxmox::Errors::NoTemplateAvailable)
  else
    template_id = found_ids.first
    template_cache[template] = template_id
    puts "Updating cache: #{template_cache}"

    # Rewind, write, and truncate to update the cache file

```

```

        file.rewind
        file.write(JSON.generate(template_cache))
        file.truncate(file.pos)

        puts "Cache written to #{cache_file_path}."
        return template_id
    end
end

end

def upload_file(file, content_type: required('content_type'), node: required('
    node'), storage: required('storage'), replace: false)
    delete_file(filename: file, content_type: content_type, node: node, storage:
        storage) if replace
    unless is_file_in_storage? filename: file, node: node, storage: storage
        res = post "/nodes/#{node}/storage/#{storage}/upload", content: content_type,
            filename: File.new(file, 'rb
                '), node: node, storage:
                    storage

        wait_for_completion task_response: res, timeout_message: 'vagrant_proxmox.
            errors.upload_timeout'
    end
end

def delete_file(filename: required('filename'), content_type: required('
    content_type'), node: required('node'), storage: required('storage'))
    delete "/nodes/#{node}/storage/#{storage}/content/#{content_type}/#{File.
        basename filename}"
end

def list_storage_files(node: required('node'), storage: required('storage'))
    res = get "/nodes/#{node}/storage/#{storage}/content"
    res[:data].map { |e| e[:volid] }
end

```

```

def get_node_ip(node, interface)
  response = get "/nodes/#{node}/network/#{interface}"
  response[:data][:address]
rescue ApiError::ServerError
  :not_created
end

# This is called every time to retrieve the node and vm_type, hence on large
# installations this could be a huge amount of data.
# @vm_info_cache is used to buffer the info we need, but maybe this should be
# read from the Vagrantfile -- do we need to get this from proxmox?

private

def get_vm_info(vm_id)
  # only look up each VM once -- and cache the results
  if @vm_info_cache.key? vm_id
    @vm_info_cache[vm_id]
    # File.write("#{Dir.home}/.vagrant_proxmox_vm_info_cache.json", JSON.dump(
      @vm_info_cache))
  else
    response = get '/cluster/resources?type=vm'
    @vm_info_cache[vm_id] = response[:data]
    .select { |m| m[:id] =~ /^[a-z]*\/#{vm_id}$/ }
    .map { |m| { id: vm_id, type: /^(.*)\/(.*)$/ .match(m[:id])[1], node: m[:
      node] } }
    .first
    # binding.irb
  end
end

private

def get_task_exitstatus(task_upid)
  node = /UPID:(.*?):/ .match(task_upid)[1]

```

```

    response = get "/nodes/#{node}/tasks/#{task_upid}/status"
    response[:data][:exitstatus]
end

private

def get(path)
  response = RestClient::Request.execute(
    method: :get,
    url: "#{api_url}#{path}",
    cookies: { PVEAuthCookie: ticket },
    verify_ssl: OpenSSL::SSL::VERIFY_NONE
  )
  JSON.parse response.to_s, symbolize_names: true
rescue RestClient::NotImplemented
  raise ApiError::NotImplemented
rescue RestClient::InternalServerError => x
  raise ApiError::ServerError, "#{x.message} for GET #{api_url}#{path} details:
    #{response.to_s} - #{x.inspect}"
rescue RestClient::Unauthorized
  raise ApiError::UnauthorizedError
rescue => x
  raise ApiError::ConnectionError, x.message
end

private

def delete(path, _params = {})
  response = RestClient::Request.execute(
    method: :delete,
    url: "#{api_url}#{path}",
    headers: headers,
    verify_ssl: OpenSSL::SSL::VERIFY_NONE
  )
  JSON.parse response.to_s, symbolize_names: true

```



```

rescue RestClient::Unauthorized
  raise ApiError::UnauthorizedError
rescue RestClient::NotImplemented
  raise ApiError::NotImplemented
rescue RestClient::InternalServerError => x
  raise ApiError::ServerError, "#{x.message} for DELETE #{api_url}#{path}"
rescue => x
  raise ApiError::ConnectionError, x.message
end

private

def post(path, params = {})
  response = RestClient::Request.execute(
    method: :post,
    url: "#{api_url}#{path}",
    payload: params,
    headers: headers,
    verify_ssl: OpenSSL::SSL::VERIFY_NONE
  )
  JSON.parse response.to_s, symbolize_names: true
rescue RestClient::Unauthorized
  raise ApiError::UnauthorizedError
rescue RestClient::NotImplemented
  raise ApiError::NotImplemented
rescue RestClient::InternalServerError => x
  raise ApiError::ServerError, "#{x.message} for POST #{api_url}#{path} details:
    #{response.to_s} - #{x.inspect}"
rescue => x
  raise ApiError::ConnectionError, x.message
end

private

def headers

```

```

    base = { content_type: 'application/x-www-form-urlencoded' }

    ticket.nil? ? base : base.merge(CSRFPventionToken: csrf_token, cookies: {
      PVEAuthCookie: ticket })
  end

  private

  def is_file_in_storage?(filename: required('filename'), node: required('node'),
    storage: required('storage'))
    (list_storage_files node: node, storage: storage).find { |f| f =~ /#{File.
      basename filename}/ }
  end
end
end
end
end

```

B.2.5 config_clone.rb

`/.vagrant.d/gems/3.3.8/gems/vagrant-proxmox-0.3.0/lib/vagrant-proxmox/action/config_clone.rb`

```

module VagrantPlugins
  module Proxmox
    module Action

      # This action modifies the configuration of a cloned vm
      # Basically it creates a user network interface with hostfwd for the provisioning
      # and an interface for every public or private interface defined in the Vagrantfile

      class ConfigClone < ProxmoxAction

        def initialize app, env
          @app = app
          @logger = Log4r::Logger.new 'vagrant_proxmox::action::config_clone'
          @node_ip = nil
          @guest_port = nil
        end
      end
    end
  end
end

```

```

end

def call env
  env[:ui].info I18n.t('vagrant_proxmox.configuring_vm')
  config = env[:machine].provider_config
  node = env[:proxmox_selected_node]
  vm_id = nil

  # Ludus templates j tm rede/bridge configurada (vibr1000) e no usamos
forwarded ports.
  # Se isto estiver ativo, saltamos a lgica de "node ip" que aqui est a
dar 400.
  if config.disable_adjust_forwarded_port
    env[:ui].detail "Skipping node ip / forwarded port adjustments (
      disable_adjust_forwarded_port=true)"
  end
  return @app.call(env)
end

env[:ui].detail "Configuring cloned VM"
begin
  vm_id = env[:machine].id&.split("/").&.last
  if vm_id.nil?
    raise VagrantPlugins::Proxmox::Errors::VMConfigError,
      error_msg: "Unable to determine VM ID"
  end
  env[:ui].detail "Getting node ip"
  @node_ip = connection(env).get_node_ip(node, 'vibr4') if config.vm_type == :qemu
  env[:ui].detail "Getting guest port"
  @guest_port = sprintf("22%03d", vm_id.to_i).to_s
  rescue StandardError => e
  raise VagrantPlugins::Proxmox::Errors::VMConfigError, error_msg: e.message
end

begin

```

```

template_config = connection(env).get_vm_config node: node, vm_id: vm_id, vm_type:
    config.vm_type
params = create_params_qemu(config, env, vm_id, template_config)
exit_status = connection(env).config_clone node: node, vm_type: config.vm_type, params
    : params
exit_status == 'OK' ? exit_status : raise(VagrantPlugins::Proxmox::Errors::
    VMConfigError, error_msg: exit_status)
    rescue StandardError => e
raise VagrantPlugins::Proxmox::Errors::VMConfigError, error_msg: e.message
    end

    env[:ui].info I18n.t('vagrant_proxmox.done')
    next_action env
end

private
def create_params_qemu(provider_config, env, vm_id, template_config)
    vm_config = env[:machine].config.vm
    params = {
vmid: vm_id,
description: "#{provider_config.vm_name_prefix}#{env[:machine].name}",
    }
    # # delete existing network interfaces from template
    # to_delete = template_config.keys.select{|key| key.to_s.match(/~net/)} }
    # params[:delete] = to_delete.join(",") if not to_delete.empty?
    # # net0 is the provisioning network, derived from forwarded_port
    # net_num = 0
    # hostname = vm_config.hostname || env[:machine].name
    # netdev0 = [
    #   "type=user",
    #   "id=net0",
    #   "hostname=#{hostname}",
    #   "hostfwd=tcp:#{@node_ip}:#{@guest_port}-:22", # selected_node's
           primary ip and port (22000 + vm_id)
    # ]

```

```

# device0 = [
#   "#{provider_config.gemu_nic_model}",
#   "netdev=net0",
#   "bus=pci.0",
#   "addr=0x12", # starting point for network interfaces
#   "id=net0",
#   "bootindex=299"
# ]
# params[:args] = "-netdev " + netdev0.join(",") + " -device " + device0
#   .join(",")
# # now add a network device for every public_network or private_network
# # ip addresses are ignored here, as we can't configure anything inside
#   the qemu vm.
# # at least we can set the predefined mac address and a bridge
# net_num += 1
# vm_config.networks.each do |type, options|
#   next if not type.match(/p.*_network$/)
#   nic = provider_config.gemu_nic_model
#   nic += "={options[:macaddress]}" if options[:macaddress]
#   nic += ",bridge={options[:bridge]}" if options[:bridge]
#   net = 'net' + net_num.to_s
#   params[net] = nic
#   net_num += 1
# end

# # some more individual settings
# params[:ide2] = "#{provider_config.gemu_iso},media=cdrom" if
#   provider_config.gemu_iso
# params[:sockets] = "#{provider_config.gemu_sockets}".to_i if
#   provider_config.gemu_sockets
# params[:cores] = "#{provider_config.gemu_cores}".to_i if
#   provider_config.gemu_cores
# params[:balloon] = "#{provider_config.vm_memory}".to_i if
#   provider_config.vm_memory and provider_config.vm_memory <
#     template_config[:balloon]

```

```

        params[:memory] = "#{provider_config.vm_memory}".to_i if provider_config
            .vm_memory
        params
    end
end

    end
end
end

```

B.2.6 clone_vm.rb

`/usr/share/vagrant.d/gems/3.3.8/gems/vagrant-proxmox-0.3.0/lib/vagrant-proxmox/action/clone_vm.rb`

```

module VagrantPlugins
  module Proxmox
    module Action

      # This action clones from a qemu template on the Proxmox server and
      # stores its node and vm_id env[:machine].id
      class CloneVm < ProxmoxAction

        def initialize app, env
          @app = app
          @logger = Log4r::Logger.new 'vagrant_proxmox::action::clone_vm'
        end

        def call env
          env[:ui].info I18n.t('vagrant_proxmox.cloning_vm')
          config = env[:machine].provider_config

          node = env[:proxmox_selected_node]
          vm_id = nil
          template_vm_id = nil
        end
      end
    end
  end
end

```

```

        begin
template_vm_id = connection(env).get_qemu_template_id(config.qemu_template)
        rescue StandardError => e
raise VagrantPlugins::Proxmox::Errors::VMCloneError, proxmox_exit_status: e.message
        end

        begin
vm_id = connection(env).get_free_vm_id
params = create_params_qemu(config, env, vm_id, template_vm_id)
exit_status = connection(env).clone_vm node: node, vm_type: config.vm_type, params:
    params
exit_status == 'OK' ? exit_status : raise(VagrantPlugins::Proxmox::Errors::
    ProxmoxTaskFailed, proxmox_exit_status: exit_status)
        rescue StandardError => e
raise VagrantPlugins::Proxmox::Errors::VMCloneError, proxmox_exit_status: e.message
        end

        env[:machine].id = "#{node}/#{vm_id}"

        env[:ui].info I18n.t('vagrant_proxmox.done')
        next_action env
    end
private
def create_params_qemu(config, env, vm_id, template_vm_id)
    { vmid: template_vm_id,
      newid: vm_id,
      name: env[:machine].config.vm.hostname || env[:machine].name.to_s,
      description: "#{config.vm_name_prefix}#{env[:machine].name}",
      full: 1 }
    end
end
        end
    end
end
end

```